

Object-Oriented Programming (OOP) for Genomics

Sergio Peignier

Contents

1	Warm-Up: Procedural vs. Object-Oriented Thinking	3
1.1	Procedural Approach (No Classes)	3
1.2	The Object-Oriented Paradigm: The <code>Sequence</code> Class	3
1.3	Prediction Questions	3
1.4	Hands-On Exercises	4
1.5	Summary	4
2	Memory, Object Identity, and Mutability	5
2.1	Motivation	5
2.2	Mutation vs Reference Copying	5
2.3	Prediction Questions	6
2.4	Hands-On Exercises	6
2.5	Summary	6
3	Python Magic Methods	7
3.1	Motivation	7
3.2	Magic Methods Syntax	7
3.3	Prediction Questions	8
3.4	Hands-On Exercises	8
3.5	Summary	8
4	Property Decorators	9
4.1	Motivation	9
4.2	Property Decorators Syntax	9
4.3	Prediction Questions	10
4.4	Hands-On Exercises	10
4.5	Summary	10
5	Specialized Method Decorators	12
5.1	Motivation	12
5.2	Code Implementation: Factory Methods & Utilities	12
5.3	Prediction Questions	13
5.4	Hands-On Questions	13
5.5	Comparison of Method Types in Python	14
6	Abstract Classes, Inheritance and Polymorphism	15
6.1	Motivation	15
6.2	Abstract Classes and Inheritance Syntax	15
6.3	Conceptual Predictions	16
6.4	Hands-On Exercises	16
6.5	Composition and Inheritance: "Has-A" vs. "Is-A" relationships	17
6.6	Summary	17

7	Multiple Inheritance and MRO	18
7.1	Motivation	18
7.2	Code Implementation: The Diamond Pattern	18
7.3	Prediction Questions	19
7.4	Hands-On Questions	19
7.5	Summary	20

1 Warm-Up: Procedural vs. Object-Oriented Thinking

Suppose we need to process biological sequences in Python without using classes. We maintain independent variables for sequence strings, metadata, and calculation functions.

1.1 Procedural Approach (No Classes)

```
1 seq1_data = "ATGCGATGA"
2 seq1_species = "Homo sapiens"
3 seq1_gene = "TP53"
4
5 def calculate_gc_content(seq_string):
6     g_count = seq_string.count("G")
7     c_count = seq_string.count("C")
8     return ((g_count + c_count) / len(seq_string)) * 100
9
10 print(f"GC%: {calculate_gc_content(seq1_data):.2f}%")
```

Discussion Questions:

1. How well does this approach scale when processing 10,000 FASTA records?
2. What stops us from accidentally passing a species name string into `calculate_gc_content()`?

1.2 The Object-Oriented Paradigm: The Sequence Class

```
1 class Sequence:
2     molecule_type = "DNA" # Class Attribute
3
4     def __init__(self, seq_id, species, seq_data):
5         self.seq_id = seq_id # Instance Attribute
6         self.species = species # Instance Attribute
7         self.seq_data = seq_data.upper()
8
9     def gc_content(self):
10        gc_count = self.seq_data.count("G") + self.seq_data.count("C")
11        return (gc_count / len(self.seq_data)) * 100
12
13    def transcribe(self):
14        return self.seq_data.replace("T", "U")
15
16 # Instantiation
17 s1 = Sequence("SEQ001", "Homo sapiens", "ATGCGATGA")
18 s2 = Sequence("SEQ002", "Mus musculus", "atgcccgcg")
```

1.3 Prediction Questions

1. What will `s2.seq_data` contain upon instantiation, given the input string "atgcccgcg"?
2. What is the result of `s1.transcribe()`? Does it mutate `s1.seq_data`?
3. What values do `s1.molecule_type` and `s2.molecule_type` return?

1.4 Hands-On Exercises

- **Exercise 1 (Validation):** Modify `__init__` to verify that `seq.data` only contains valid nucleotides ($\{A, T, C, G\}$).
- **Exercise 2 (New Behavior):** Implement a method `reverse_complement(self)` returning the inverted complementary strand.
- **Exercise 3 :** Try setting `Sequence.molecule_type = "RNA"`. What happens to `s1.molecule_type` and `s2.molecule_type`? Instantiate a new object belonging to the `Sequence` class, what happens?

1.5 Summary

Syntax Element	Role in Genomics Context
<code>class Sequence</code>	Defines the blueprint for storing and manipulating any sequence.
<code>__init__()</code>	Constructor method run automatically when instantiated.
<code>self</code>	References the specific instance (e.g., <code>s1</code> vs <code>s2</code>).
Instance Attribute	Variable tied to one instance (e.g., <code>self.seq.data</code>).
Class Attribute	Variable shared by all instances (e.g., <code>molecule_type = "DNA"</code>).

Table 1: Key Python OOP Syntax Components

2 Memory, Object Identity, and Mutability

2.1 Motivation

In computational biology, sequence alignment algorithms and variant callers often process millions of reads. A common architectural bug occurs when multiple variables or objects point to the same underlying sequence data in memory. If one process mutates the sequence in place (e.g., masking low-quality bases to N), all other references pointing to that same memory location are silently altered.

Understanding the difference between object value equality (`==`), memory identity (`is`), and mutability (in-place modification vs. creating new copies) is critical for writing memory-safe genomic pipelines.

2.2 Mutation vs Reference Copying

```
1
2 import copy
3
4 # 1. MUTABLE MUTATION IN-PLACE (Lists)
5 class MutableDNA:
6     def __init__(self, seq_id, bases):
7         self.seq_id = seq_id
8         self.bases = list(bases) # Bases stored as a mutable list
9
10    def mask_low_quality(self, start, end):
11        # Mutates the list IN PLACE
12        for i in range(start, end):
13            self.bases[i] = "N"
14
15
16 # 2. IMMUTABLE DESIGN (Strings)
17 class ImmutableDNA:
18     def __init__(self, seq_id, bases):
19         self.seq_id = seq_id
20         self.bases = str(bases) # Bases stored as an immutable string
21
22    def mask_low_quality(self, start, end):
23        # Creates and returns a NEW instance, leaving original intact
24        new_bases = self.bases[:start] + "N" * (end - start) + self.
bases[end:]
25        return ImmutableDNA(f"{self.seq_id}_masked", new_bases)
26
27
28 # --- EXECUTING THE SCENARIOS ---
29
30 # Scenario A: Shallow Reference Copying
31 read_1 = MutableDNA("READ_01", "ATGCGATGA")
32 read_2 = read_1 # Reference copy (alias)
33
34 # Scenario B: Separate Instances with Identical Data
35 read_3 = MutableDNA("READ_01", "ATGCGATGA")
36
37 # Scenario C: Immutable Pattern
38 immutable_1 = ImmutableDNA("READ_02", "ATGCGATGA")
39 immutable_2 = immutable_1.mask_low_quality(0, 3)
```

2.3 Prediction Questions

1. **"is" vs "==" on References:** What will `read_1 is read_2` evaluate to? What happens to `read_2.bases` after executing `read_1.mask_low_quality(0, 3)`?
2. **"is" vs "==" on Independent Objects:** What will `read_1 == read_3` evaluate to (assuming default Python behavior)? What will `read_1 is read_3` evaluate to?
3. **Immutable Creation:** Does executing `immutable_1.mask_low_quality(0, 3)` change `immutable_1.bases`? What will `immutable_1 is immutable_2` evaluate to?

2.4 Hands-On Exercises

- **Exercise 1 (Fixing the Alias Bug with Deep Copies):** Modify Scenario A using Python's `copy.deepcopy()` so that `read_2` receives a distinct copy of `read_1`. Verify that `read_2 is read_1` is `False` and that mutating `read_1` no longer alters `read_2`.
- **Exercise 2 (Exploring String Interning):** In Python, small strings are often interned in memory. Run `s1 = "ATGC"`, `s2 = "ATGC"`, and test `s1 is s2`. Now try with longer strings dynamically built at runtime (`s3 = "".join(list(s2))`). Explain why `s1 is s3` may evaluate to `False`.
- **Exercise 3 (Designing an In-Place Transversion Method):** Write a method `mutate_base(self, position, new_base)` for `MutableDNA`. Ensure it validates that `position` is within range before performing the in-place replacement.

2.5 Summary

Mutable Objects (e.g., `list`, `dict`, `set`) can be modified in place. Passing a mutable sequence to multiple analysis functions risks unintended side effects if any function alters the data. Immutable Objects (e.g., `str`, `tuple`, `frozenset`) cannot be altered after creation. Operations that transform the sequence (e.g., complementation, slicing, masking) must construct and return a new instance. When building genomic classes that manage lists or dictionary annotations internally, pass copies during initialization to prevent external references from mutating internal state.

Operator/Concept	Evaluation Mechanism	Pipeline Impact
<code>a is b</code>	<code>id(a) == id(b)</code>	Verifies exact memory aliasing.
<code>a == b</code>	<code>a.__eq__(b)</code>	Verifies equality of sequence content. If a class doesn't define <code>__eq__</code> function (check next section), it inherits the default from object, which delegates to identity so <code>==</code> behaves exactly like <code>is</code>
Mutable	In-place state updates	Risk of unintended side effects across references.
Immutable	Returns new instances	Thread-safe, side-effect-free pipeline transformations.

Table 2: Memory Management Fundamentals in Python

3 Python Magic Methods

3.1 Motivation

We have instantiated our `DNASequences` object in Python. Now, an analyst tries to interact with it using standard Python operators and built-in functions

```
1 s1 = DNASequences("SEQ01", "ATGCGATGA")
2
3 # Problem 1: Unfriendly String Representation
4 print(s1)
5 # Output: <__main__.DNASequences object at 0x7f8a102b4190> (Useless for
   debugging!)
6
7 # Problem 2: Built-in Python functions fail
8 len(s1)                # TypeError: object of type 'DNASequences'
   has no len()
9 "ATG" in s1            # TypeError: argument of type 'DNASequences'
   is not iterable
10 s1[0:3]                # TypeError: 'DNASequences' object is not
   subscriptable
11
12 # Problem 3: Value equality check fails
13 s2 = DNASequences("SEQ01", "ATGCGATGA")
14 s1 == s2               # Evaluates to False! (Checks memory address
   , not sequence content)
```

Discussion Questions

1. Why does Python treat custom objects like `s1` as "opaque" entities until we explicitly define special methods?
2. What is the difference between making an object look nice in a log file (`__repr__`) versus displaying it to a user (`__str__`)?
3. How can we make our `DNASequences` class behave like native Python data structures (such as strings or lists) so that operations like `len()`, `[]`, and `in` work seamlessly?

3.2 Magic Methods Synthesis

```
1 class DNASequences:
2     def __init__(self, seq_id, sequence):
3         self.seq_id = seq_id
4         self.sequence = sequence.upper()
5
6     def __str__(self):
7         return f">{self.seq_id}\n{self.sequence}"
8
9     def __repr__(self):
10        return f"DNASequences(seq_id={self.seq_id!r}, sequence={self.
sequence!r})"
11
12    def __len__(self):
13        return len(self.sequence)
14
15    def __getitem__(self, index):
16        if isinstance(index, slice):
17            return DNASequences(f"{self.seq_id}_slice", self.sequence[
index])
```

```

18         return self.sequence[index]
19
20     def __contains__(self, subseq):
21         return subseq.upper() in self.sequence
22
23     def __eq__(self, other):
24         if not isinstance(other, DNASSequence):
25             return False
26         return self.seq_id == other.seq_id and self.sequence == other.
sequence

```

3.3 Prediction Questions

1. **__repr__ vs __str__:** What will `print(seq1)` output versus inspecting `seq1` directly in an interactive Python shell (or running `repr(seq1)`)? Remove the `__str__` method, instantiate `seq1` again and use the `print` function, what happens?
2. **__getitem__ with slices:** What is the result and type of `sub = seq1[0:4]`? What will `sub.seq_id` return?
3. **__eq__ vs is:** What will `seq1 == seq2` evaluate to? What about `seq1 is seq2`?

3.4 Hands-On Exercises

- **Exercise 1 (Sequence Addition – __add__):** Implement the `__add__(self, other)` magic method so that `seq1 + seq2` returns a new `DNASSequence` object with concatenated sequences and a combined ID (e.g., "SEQ01+SEQ02").
- **Exercise 2 (String Formatting – __format__):** Add a `__format__(self, format_spec)` method so that `f"{seq1:fasta}"` formats it as a FASTA string, while `f"{seq1:stats}"` returns `SEQ01 (Length: 9 bp)`.

3.5 Summary

Magic methods are special predefined methods in Python surrounded by double underscores (`__method__`). They allow custom classes to hook directly into Python's built-in operators, protocols, and language syntax.

Magic Method	Trigger	Role in Genomic Data Structures
<code>__str__</code>	<code>print(obj)</code>	Returns user-facing sequence string (e.g., FASTA layout).
<code>__repr__</code>	<code>repr(obj)</code>	Unambiguous representation for debugging and logging.
<code>__len__</code>	<code>len(obj)</code>	Returns total sequence length in base pairs.
<code>__getitem__</code>	<code>obj[i:j]</code>	Enables nucleotide indexing and sub-sequence slicing.
<code>__contains__</code>	<code>mot in obj</code>	Checks motif/primer presence within the sequence.
<code>__eq__</code>	<code>obj1 == obj2</code>	Evaluates structural equality based on attributes.

Table 3: Key Python Dunder Methods for Computational Biology

4 Property Decorators

4.1 Motivation

In our genomics pipeline, we have a `DNASequence` class. Analysts frequently interact with two key properties: `sequence`: The raw nucleotide string, `gc_content`: A calculated metric based on the sequence. Without encapsulation, attributes are public and exposed directly:

```
1 # The Naive Approach
2 class UnprotectedSequence:
3     def __init__(self, seq_id, sequence):
4         self.seq_id = seq_id
5         self.sequence = sequence
6
7 s1 = UnprotectedSequence("SEQ01", "ATGC")
8
9 # Problem 1: Unrestricted Mutation / Invalid Data
10 s1.sequence = "ATGCZ123!!" # Corrupted sequence accepted without
    warning!
11
12 # Problem 2: Stale Computed Data
13 s1.gc_content = 50.0
14 s1.sequence = "CCCC"      # Sequence changed, but gc_content is still
    50.0!
```

Discussion Questions

1. **Validation:** How can we prevent an analyst or pipeline step from assigning non-nucleotide characters (Z, 1, !) to `.sequence`?
2. **Maintenance:** If we write traditional methods like `get_sequence()` and `set_sequence()`, anyone using our code has to change their existing scripts from `obj.sequence` to `obj.get_sequence()`. How can we add validation without breaking existing code syntax?
3. **Resource Cleanup:** What should happen if a sequence is deleted from memory? Should we log its removal?

4.2 Property Decorators Syntax

```
1 class DNASequence:
2     def __init__(self, seq_id, sequence):
3         self.seq_id = seq_id
4         self._sequence = None
5         self.sequence = sequence # Delegates to setter
6
7     @property
8     def sequence(self):
9         """Getter: Access protected sequence string."""
10        return self._sequence
11
12    @sequence.setter
13    def sequence(self, new_seq):
14        """Setter: Validates input before assignment."""
15        new_seq = new_seq.upper()
16        if not set(new_seq).issubset(set("ATCGN")):
17            raise ValueError(f"Invalid nucleotides in: {new_seq}")
18        self._sequence = new_seq
19
```

```

20     @property
21     def gc_content(self):
22         """Computed Property: Getter only."""
23         gc = self._sequence.count("G") + self._sequence.count("C")
24         return (gc / len(self._sequence)) * 100
25
26     @sequence.deleter
27     def sequence(self):
28         """Deleter: Cleanup hook."""
29         print(f"Logging deletion of sequence in {self.seq_id}")
30         del self._sequence

```

4.3 Prediction Questions

1. Why does `__init__` assign to `self.sequence` instead of `self._sequence`?
2. Question 2 (Computed Property): If we try to manually override the GC content using `seq1.gc_content = 60.0`, what will Python do?
3. How does Python handle `seq.sequence = "ATGCZ"`? Does `self._sequence` retain its former valid state?

4.4 Hands-On Exercises

- **Exercise 1 (Read-Only Attribute):** Add a `@property` for `quality_score_mean`. It should calculate and return the average score from a private list `self._quality_scores`. Do not provide a `@setter` for it so that it remains strictly read-only.
- **Exercise 2 (Setter Range Validation):** Create a property `phred_threshold` with a setter that only accepts integers between 0 and 40. If a value outside this range is assigned, raise a `ValueError`.
- **Exercise 3 (Deleter Safety Check):** Modify `@sequence.deleter` so that if someone attempts `del seq1.sequence`, it checks a boolean attribute `self.is_locked`. If `is_locked == True`, prevent deletion and print "Deletion blocked: Sequence is locked!".

4.5 Summary

Encapsulation Principle and Python's Convention: Unlike languages with strict private keywords (Java, C++), Python uses naming conventions to signal privacy:

- `public_variable`: Accessible and editable by anyone.
- `_protected_variable` (single underscore): Signals "Internal implementation detail—do not modify directly outside this class."
- `__private_variable` (double underscore): Triggers name mangling to prevent accidental overwrites

Decorator	Function in Genomics Application
<code>@property</code>	Defines getter or dynamic computed field (e.g., <code>gc_content</code>). Prevents direct overwrite if no setter exists (the attribute becomes read-only). Syntactically accessed like a variable (<code>seq1.sequence</code>), but executes a function under the hood.
<code>@<attr>.setter</code>	Intercepts modifications to validate data integrity (e.g., checking nucleotide alphabets) before saving. Preserves backward compatibility: you can convert a public attribute into a managed property without changing external code calling <code>obj.attr</code>
<code>@<attr>.deleter</code>	Executes custom logging or memory release logic when <code>del obj.attr</code> is called.

Table 4: Python Property Decorators Summary

5 Specialized Method Decorators

5.1 Motivation

In our genomics pipeline, we have a `DNASequences` class. As our software expands, we run into two common structural challenges:

- **Alternative Constructors / Factory Creation:** Sequence data often comes from external files in different formats—such as FASTA strings or GenBank records. Right now, `__init__` only accepts parsed arguments (`seq_id`, `species`, `sequence`).
- **Utility Helper Functions:** We need a helper function to validate whether a string contains valid nucleotides or calculate complement bases. This function is useful even when we don't have a specific `DNASequences` object instantiated yet.

In this case the implementation would look like:

```
1 # The Naive Approach
2 class DNASequences:
3     def __init__(self, seq_id, species, sequence):
4         self.seq_id = seq_id
5         self.species = species
6         self.sequence = sequence
7
8 # Problem 1: Parsing code must live OUTSIDE the class, polluting the
9 # global namespace
10 def parse_fasta_string(fasta_text):
11     lines = fasta_text.strip().split("\n")
12     header = lines[0][1:].split(" ") # Remove '>'
13     seq_id, species = header[0], header[1]
14     sequence = "".join(lines[1:])
15     return DNASequences(seq_id, species, sequence)
16
17 # Problem 2: Standalone helper functions have no organizational tie to
18 # DNASequences
19 def is_valid_dna(sequence):
20     return set(sequence.upper()).issubset(set("ATCGN"))
```

Discussion Questions

1. **Namespace and Organization:** Why is it cleaner to bundle `parse_fasta_string()` inside the `DNASequences` class rather than keeping it as a loose global function?
2. **Class State:** If we want to track a global counter (e.g., `total_sequences_created`) across all instances, how can a method modify that class-level variable cleanly?
3. **Instance Independence:** Does a utility function like `is_valid_dna("ATCG")` need access to a specific instance (`self`) to do its job?

5.2 Code Implementation: Factory Methods & Utilities

```
1 class DNASequences:
2     # Class Attribute (shared across all instances)
3     total_sequences_created = 0
4     default_organism = "Unknown Species"
5
6     def __init__(self, seq_id, species, sequence):
7         self.seq_id = seq_id
8         self.species = species
9         self.sequence = sequence.upper()
10         # Increment class counter every time an instance is made
```

```

11         DNASquence.total_sequences_created += 1
12
13     # 1. CLASS METHOD (@classmethod) - Alternative Constructor
14     @classmethod
15     def from_fasta(cls, fasta_str):
16         """Factory method: creates a DNASquence object from a FASTA
17         string."""
18         lines = fasta_str.strip().split("\n")
19         header = lines[0].lstrip(">").split("|")
20         seq_id = header[0]
21         species = header[1] if len(header) > 1 else cls.default_organism
22         sequence = "".join(lines[1:])
23
24         # cls(...) instantiates a new object of THIS class
25         return cls(seq_id, species, sequence)
26
27     # 2. CLASS METHOD (@classmethod) - Modifying Class State
28     @classmethod
29     def set_default_organism(cls, new_organism):
30         """Modifies class-level default settings for future sequence
31         creations."""
32         cls.default_organism = new_organism
33
34     # 3. STATIC METHOD (@staticmethod) - Standalone Utility
35     @staticmethod
36     def is_valid_dna(sequence_str):
37         """Pure utility: checks validity without needing 'self' or 'cls'
38         ."""
39         valid_bases = set("ATCGN")
40         return set(sequence_str.upper()).issubset(valid_bases)
41
42 # Standard Instantiation
43 seq1 = DNASquence("SEQ01", "Homo sapiens", "ATGCGATGA")
44
45 # Alternative Instantiation via Class Method
46 fasta_data = ">SEQ02|Mus musculus\nATGCCC\nGCGG"
47 seq2 = DNASquence.from_fasta(fasta_data)

```

5.3 Prediction Questions

1. What is the advantage of using `return cls(...)` in `from_fasta` instead of `return DNASquence(...)` when working with subclasses?
2. Can `DNASquence.is_valid_dna("ATGC")` be executed before any instance of `DNASquence` is created? Why or why not? What will it return?

5.4 Hands-On Questions

- **Exercise 1 (New @classmethod Factory):** Add a class method `from_dict(cls, data_dict)` that takes a dictionary (e.g., `"id": "SEQ03", "species": "E. coli", "seq": "GATC"`) and returns a new instance of the class.
- **Exercise 2 (New @staticmethod Utility):** Add a static method `gc_ratio(sequence_str)` that takes a raw string, calculates its GC percentage, and returns it as a float. Ensure it can be called on raw strings without instantiating any object.

- **Exercise 3 (Class State and Defaults):** Call `DNASequence.set_default_organism("Synthetic Construct")`. Create a sequence using a FASTA string without a species header. Verify that its species attribute is set to "Synthetic Construct".

5.5 Comparison of Method Types in Python

- For Class Methods (`@classmethod`) the first argument is automatically passed as `cls` (referencing the class itself, not an instance). These methods have two advantages: i) Key Advantage 1 (Factory Pattern), it allows multiple ways to construct an object beyond the single `__init__` method. ii) Key Advantage 2 (Inheritance Friendly), using `cls(...)` inside a class method guarantees that child classes inherit the factory correctly and instantiate their own type rather than the parent type.
- Static Methods (`@staticmethod`) take no implicit first argument (self or cls). It functions as a standard utility function that lives inside the class's namespace for organizational clarity. Their key advantage is that they communicate to other developers that this method is pure and stateless—it does not modify or depend on instance or class attributes.

Method Type	Decorator	1st Arg	State Access	Primary Purpose Here
Instance Method	None	<code>self</code>	Instance & Class	Calculations on object data (<code>gc_content</code>).
Class Method	<code>@classmethod</code>	<code>cls</code>	Class State Only	Alternative constructors (<code>from_fasta</code>).
Static Method	<code>@staticmethod</code>	None	No State	Pure utilities (<code>is_valid_dna</code>).

Table 5: Python Method Types Comparison

6 Abstract Classes, Inheritance and Polymorphism

6.1 Motivation

Our genomics pipeline now needs to process **two distinct biological molecules**: DNA sequences and Protein sequences.

- Both have an identifier (`seq_id`) and a species name (`species`).
- DNA has a `gc_content()` method and a `transcribe()` method.
- Protein has an `is_hydrophobic()` method and a `molecular_weight()` method.

We want to write a single pipeline function `process_record(seq)` that takes any sequence object, validates it, and prints its summary regardless of whether it is DNA or Protein.

Discussion Questions

1. If we keep creating completely separate `DNASequence` and `ProteinSequence` classes from scratch, how much code do we end up duplicating?
2. How can we force every biological sequence type in our system to implement a required method (like `validate()`) so our pipeline never crashes on an unexpected sequence type?
3. What happens if someone tries to instantiate a generic "Sequence" that isn't DNA, RNA, or Protein? Does a generic sequence exist in nature?

6.2 Abstract Classes and Inheritance Syntax

```
1 from abc import ABC, abstractmethod
2
3 # 1. ABSTRACT BASE CLASS (Abstraction)
4 class BiologicalSequence(ABC):
5     def __init__(self, seq_id, species, sequence):
6         self.seq_id = seq_id
7         self.species = species
8         self.sequence = sequence.upper()
9
10    @abstractmethod
11    def validate(self):
12        """Every child class MUST implement its own validation rule."""
13        pass
14
15    @abstractmethod
16    def get_summary(self):
17        """Every child class MUST return a formatted summary string."""
18        pass
19
20
21 # 2. INHERITED CHILD CLASSES (Inheritance & Polymorphism)
22 class DNASequence(BiologicalSequence):
23     def validate(self):
24         valid_bases = set("ATCG")
25         return set(self.sequence).issubset(valid_bases)
26
27     def gc_content(self):
28         gc = self.sequence.count("G") + self.sequence.count("C")
29         return (gc / len(self.sequence)) * 100
```

```

30
31     def get_summary(self):
32         return f"DNA [{self.seq_id}] | Species: {self.species} | GC: {
self.gc_content():.1f}%"
33
34
35 class ProteinSequence(BiologicalSequence):
36     def validate(self):
37         # Basic check for standard 20 amino acid single-letter codes
38         invalid_chars = set("BJOUXZ")
39         return not any(char in invalid_chars for char in self.sequence)
40
41     def get_summary(self):
42         return f"Protein [{self.seq_id}] | Species: {self.species} |
Length: {len(self.sequence)} aa"
43
44
45 # 3. POLYMORPHIC PIPELINE FUNCTION
46 def run_qc_pipeline(sequence_object):
47     if sequence_object.validate():
48         print(f"VALID: {sequence_object.get_summary()}")
49     else:
50         print(f"INVALID SEQUENCE DETECTED in {sequence_object.seq_id}!")
51
52
53 # Instantiation
54 dna_sample = DNASequence("DNA01", "H. sapiens", "ATGCGATGA")
55 prot_sample = ProteinSequence("PROT01", "E. coli", "MKTIIALSYIFCLVF")

```

6.3 Conceptual Predictions

1. **Abstraction:** What happens if we execute `s = BiologicalSequence("01", "Homo sapiens", "ATGC")`? Why?
2. **Polymorphism:** How does `run_qc_pipeline()` process both `DNASequence` and `ProteinSequence` objects without checking their explicit types with `if/else`?
3. **Inheritance and Validation:** What will be the exact output if we run `run_qc_pipeline(DNASequence("DNA02", "H. sapiens", "ATGCZ"))`?

6.4 Hands-On Exercises

- **Exercise 1 (New Inheritance Class):** Create a new child class `RNASequence` that inherits from `BiologicalSequence`. Implement its `validate()` method (checking for A, U, C, G) and its `get_summary()` method.
- **Exercise 2 (Polymorphic Method Expansion):** Add an abstract method `@abstractmethod def molecular_weight(self)` to `BiologicalSequence`. Modify both `DNASequence` and `ProteinSequence` so the code runs again.
- **Exercise 3 (Pipeline Modification):** Write a function `process_batch(sequence_list)` that takes a list containing mixed DNA, RNA, and Protein objects, validates them all, and prints their summaries using a single for loop.

6.5 Composition and Inheritance: "Has-A" vs. "Is-A" relationships

When designing object-oriented software, one of the most critical architectural decisions is choosing between inheritance and composition. Inheritance models an "Is-A" relationship, where a derived class is a specialized type of its parent—such as a `DNARRead` is a specific kind of `BiologicalSequence`. However, beginners often misuse inheritance to share code between loosely related entities, leading to brittle, overly complex class hierarchies. In contrast, composition models a "Has-A" relationship, where complex domain concepts are built by combining independent, modular objects. For instance, rather than having a `Gene` inherit directly from a raw sequence class, it is conceptually clearer to say a `Gene` has a `DNASequence`, has a list of `Exon` features, and has an `Annotation` record.

Composition achieves flexibility by delegating specific responsibilities to distinct, single-purpose classes rather than stuffing all functionality into one giant parent class. In a genomics pipeline, an independent `DNASequence` object handles string operations and GC-content calculations, while a separate `QualityScore` object manages Phred metrics and filtering logic. A higher-level `FastqRecord` class can then embed instances of both as internal attributes. This decouples individual bioinformatic components: if your quality-score representation changes from Illumina to PacBio scale, you only update the `QualityScore` class, leaving `DNASequence` and `FastqRecord` entirely untouched. By favoring composition over inheritance, you build modular, easily testable pipelines where objects act as building blocks rather than rigid family trees.

6.6 Summary

Concept	Genomic Pipeline Definition
Abstraction	Enforcing a common interface (<code>@abstractmethod</code>) across all sequence types while preventing direct instantiation of non-specific sequences.
Inheritance	Reusing common fields (<code>seq_id</code> , <code>species</code>) from <code>BiologicalSequence</code> in child classes (<code>DNASequence</code> , <code>ProteinSequence</code>).
Polymorphism	Invoking <code>validate()</code> or <code>get_summary()</code> on diverse biological objects seamlessly inside unified pipeline functions.

Table 6: The Three Pillars of Advanced OOP in Computational Biology

7 Multiple Inheritance and MRO

7.1 Motivation

We want to design a modular genomic analysis pipeline using multiple inheritance structured as a diamond:

- Base Class (*A*): `Sequence` → Provides the core sequence infrastructure (`seq_id`, `sequence`).
- Branch 1 (*B*): `QualitySequence(Sequence)` → Adds Phred quality scores (FASTQ style) and overrides `process()`.
- Branch 2 (*C*): `AnnotatedSequence(Sequence)` → Adds genomic annotations (e.g., features, genes) and overrides `process()`.
- Derived Class (*D*): `FastqAnnotationRecord(QualitySequence, AnnotatedSequence)` → Combines both quality metrics and annotations.

Discussion Questions

1. If we call `process()` on a `FastqAnnotationRecord` instance, which method gets executed first: `QualitySequence`'s or `AnnotatedSequence`'s?
2. When `super().process()` is called inside `QualitySequence`, which parent class does Python call next?
3. Without a well-defined resolution order, do we risk running the base class `Sequence.process()` method twice?

7.2 Code Implementation: The Diamond Pattern

```
1 # 1. BASE CLASS (A)
2 class Sequence:
3     def __init__(self, seq_id, sequence, **kwargs):
4         print("-> Entering Sequence.__init__")
5         self.seq_id = seq_id
6         self.sequence = sequence.upper()
7         super().__init__(**kwargs) # Passes remaining arguments up to
object
8
9     def process(self):
10        print(f"[{self.seq_id}] Processing base sequence ({len(self.
sequence)}) bp")
11
12
13 # 2. BRANCH B
14 class QualitySequence(Sequence):
15     def __init__(self, quality_scores, **kwargs):
16         print("-> Entering QualitySequence.__init__")
17         self.quality_scores = quality_scores
18         super().__init__(**kwargs)
19
20     def process(self):
21         print("-> Executing QualitySequence.process(): Filtering by
Phred quality...")
22         super().process()
23
24
```

```

25 # 3. BRANCH C
26 class AnnotatedSequence(Sequence):
27     def __init__(self, annotation, **kwargs):
28         print("-> Entering AnnotatedSequence.__init__")
29         self.annotation = annotation
30         super().__init__(**kwargs)
31
32     def process(self):
33         print("-> Executing AnnotatedSequence.process(): Parsing
34 genomic annotations...")
35         super().process()
36
37 # 4. DERIVED CLASS / DIAMOND (D)
38 class FastqAnnotationRecord(QualitySequence, AnnotatedSequence):
39     def __init__(self, seq_id, sequence, quality_scores, annotation):
40         print("-> Entering FastqAnnotationRecord.__init__")
41         super().__init__(
42             seq_id=seq_id,
43             sequence=sequence,
44             quality_scores=quality_scores,
45             annotation=annotation
46         )
47
48     def process(self):
49         print("=== STARTING COMPLETE PIPELINE PROCESSING ===")
50         super().process()
51
52 # Instantiation
53 record = FastqAnnotationRecord(
54     seq_id="SEQ_FASTQ_01",
55     sequence="ATGCGATGA",
56     quality_scores=[30, 35, 28, 40, 39, 32, 31, 38, 40],
57     annotation="Exon 1 - TP53"
58 )
59 )

```

7.3 Prediction Questions

1. **__init__ Order:** In what exact order will the -> Entering ... messages print during the instantiation of record?
2. **process() Flow:** When we run record.process(), will AnnotatedSequence.process() execute? Why or why not?
3. **Method Resolution Order (MRO) Order:** What will FastqAnnotationRecord.__mro__ or FastqAnnotationRecord.mro() return?

7.4 Hands-On Questions

- **Question 1 (Inheritance Order Swap):** Change the class definition of FastqAnnotationRecord to class FastqAnnotationRecord(AnnotatedSequence, QualitySequence):. How does the MRO change, and how does that affect the output of record.process()?
- **Question 2 (Breaking the Chain):** Remove the super().process() call from inside QualitySequence.process(). What happens when you run record.process()? Does AnnotatedSequence.process() still execute?

- **Question 3 (Extending the Diamond):** Create a new child class `FilteredRecord` that inherits from `FastqAnnotationRecord`. Add a `process()` method that calls `super().process()` and print the MRO of `FilteredRecord`.

7.5 Summary

- **Method Resolution Order:** The MRO is the deterministic order Python follows to look up methods or attributes across a class hierarchy (especially under multiple inheritance). It guarantees that in complex structures (like the diamond problem), base classes are not visited before their derived classes and methods are not executed redundantly.
- **The C3 Linearization Algorithm** Python calculates the MRO using the C3 Linearization algorithm, which maintains three critical guarantees: i) **Preserving Local Order:** If `D` inherits from `(B, C)`, Python will always search `B` before `C`. ii) **Monotonicity:** If class `B` precedes `A` in search order, that relationship remains consistent across all subclasses. iii) **Single Visit Guarantee:** A common base class (such as `Sequence`) is visited exactly once and always after its derived classes.
- **3. The `super()` Function:** `super()` does not simply call the immediate parent class. In Python, `super()` delegates to the next class in the MRO chain of the current instance. Combining `super()` with `**kwargs` enables **cooperative multiple inheritance**: each class in the chain processes its own parameters/responsibilities and hands control off to the next class in the MRO.

Concept	Role in Python & Genomics Pipelines
MRO	Lookup order Python follows to resolve methods. Resolves ambiguity in diamond inheritance ($D \rightarrow B \rightarrow C \rightarrow A$).
C3 Linearization	Algorithm computing the MRO to guarantee monotonicity and single-visit base class invocation.
<code>super()</code>	Dynamic proxy delegating to the next method in the instance's MRO, enabling cooperative inheritance.

Table 7: Multiple Inheritance Mechanics in Python